

```
!pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.19.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.5.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (2025.2)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.34.4)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.2)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (4.0.0a1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (4.12.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0) (1.1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2) (2025.11.11)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.20.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2) (1.17.0)
```

```
# Transformers installation
```

```
! pip install transformers datasets
```

```
# To install from source instead of the last release, comment the command above and uncomment the following one.
```

```
# ! pip install git+https://github.com/huggingface/transformers.git
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.56.1)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.19.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.34.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.5.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (2025.2)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (4.0.0a1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (4.12.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (1.1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025.11.11)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (25.3.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (6.1.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1) (1.20.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2) (1.17.0)
```

Summary of the tasks

This page shows the most frequent use-cases when using the library. The models available allow for many different configurations and a great versatility in use-cases. The most simple ones are presented here, showcasing usage for tasks such as question answering,

sequence classification, named entity recognition and others.

These examples leverage auto-models, which are classes that will instantiate a model according to a given checkpoint, automatically selecting the correct model architecture. Please check the [AutoModel](#) documentation for more information. Feel free to modify the code to be more specific and adapt it to your specific use-case.

In order for a model to perform well on a task, it must be loaded from a checkpoint corresponding to that task. These checkpoints are usually pre-trained on a large corpus of data and fine-tuned on a specific task. This means the following:

- Not all models were fine-tuned on all tasks. If you want to fine-tune a model on a specific task, you can leverage one of the `run_$TASK.py` scripts in the [examples](#) directory.
- Fine-tuned models were fine-tuned on a specific dataset. This dataset may or may not overlap with your use-case and domain. As mentioned previously, you may leverage the [examples](#) scripts to fine-tune your model, or you may create your own training script.

In order to do an inference on a task, several mechanisms are made available by the library:

- Pipelines: very easy-to-use abstractions, which require as little as two lines of code.
- Direct model use: Less abstractions, but more flexibility and power via a direct access to a tokenizer (PyTorch/TensorFlow) and full inference capacity.

Both approaches are showcased here.

All tasks presented here leverage pre-trained checkpoints that were fine-tuned on specific tasks. Loading a checkpoint that was not fine-tuned on a specific task would load only the base transformer layers and not the additional head that is used for the task, initializing the weights of that head randomly.

This would produce random output.

✓ Sequence Classification

Sequence classification is the task of classifying sequences according to a given number of classes. An example of sequence classification is the GLUE dataset, which is entirely based on that task. If you would like to fine-tune a model on a GLUE sequence classification task, you may leverage the [run_glue.py](#), [run_tf_glue.py](#), [run_tf_text_classification.py](#) or [run_xnli.py](#) scripts.

Here is an example of using pipelines to do sentiment analysis: identifying if a sequence is positive or negative. It leverages a fine-tuned model on sst2, which is a GLUE task.

This returns a label ("POSITIVE" or "NEGATIVE") alongside a score, as follows:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I hate you")[0]
print(f"label: {result['label']}, with score: {round(result['score'], 4)}")
```

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f (https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english)
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as an environment variable or add it to your Colab secrets.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% 629/629 [00:00<00:00, 24.9kB/s]
model.safetensors: 100% 268M/268M [00:05<00:00, 57.8MB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.90kB/s]
vocab.txt: 232k/? [00:00<00:00, 3.58MB/s]
Device set to use cuda:0
label: NEGATIVE, with score: 0.9991
```

```
result = classifier("I love you")[0]
print(f"label: {result['label']}, with score: {round(result['score'], 4)}")
```

```
label: POSITIVE, with score: 0.9999
```

Here is an example of doing a sequence classification using a model to determine if two sequences are paraphrases of each other. The process is the following:

1. Instantiate a tokenizer and a model from the checkpoint name. The model is identified as a BERT model and loads it with the weights stored in the checkpoint.

2. Build a sequence from the two sentences, with the correct model-specific separators, token type ids and attention masks (which will be created automatically by the tokenizer).
3. Pass this sequence through the model so that it is classified in one of the two available classes: 0 (not a paraphrase) and 1 (is a paraphrase).
4. Compute the softmax of the result to get probabilities over the classes.
5. Print the results.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased-finetuned-mrpc")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-cased-finetuned-mrpc")

classes = ["not paraphrase", "is paraphrase"]

sequence_0 = "The company HuggingFace is based in New York City"
sequence_1 = "Apples are especially bad for your health"
sequence_2 = "HuggingFace's headquarters are situated in Manhattan"

# The tokenizer will automatically add any model specific separators (i.e. <CLS> and <SEP>) and tokens to
# the sequence, as well as compute the attention masks.
paraphrase = tokenizer(sequence_0, sequence_2, return_tensors="pt")
not_paraphrase = tokenizer(sequence_0, sequence_1, return_tensors="pt")

paraphrase_classification_logits = model(**paraphrase).logits
not_paraphrase_classification_logits = model(**not_paraphrase).logits

paraphrase_results = torch.softmax(paraphrase_classification_logits, dim=1).tolist()[0]
not_paraphrase_results = torch.softmax(not_paraphrase_classification_logits, dim=1).tolist()[0]

# Should be paraphrase
for i in range(len(classes)):
    print(f"{classes[i]}: {int(round(paraphrase_results[i] * 100))}%")
```

```
tokenizer_config.json: 100%          49.0/49.0 [00:00<00:00, 2.08kB/s]
config.json: 100%                  433/433 [00:00<00:00, 22.4kB/s]
vocab.txt: 100%                    213k/213k [00:00<00:00, 7.60MB/s]
tokenizer.json: 100%              436k/436k [00:00<00:00, 5.71MB/s]
model.safetensors: 100%          433M/433M [00:09<00:00, 60.4MB/s]
not paraphrase: 10%
is paraphrase: 90%
```

```
# Should not be paraphrase
for i in range(len(classes)):
    print(f"{classes[i]}: {int(round(not_paraphrase_results[i] * 100))}%")
```

```
not paraphrase: 94%
is paraphrase: 6%
```

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification # Changed to AutoModelForSequenceClassification
import torch # Changed to import torch

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased-finetuned-mrpc")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-cased-finetuned-mrpc") # Changed to AutoModelForSequenceClassification

classes = ["not paraphrase", "is paraphrase"]

sequence_0 = "The company HuggingFace is based in New York City"
sequence_1 = "Apples are especially bad for your health"
sequence_2 = "HuggingFace's headquarters are situated in Manhattan"

# The tokenizer will automatically add any model specific separators (i.e. <CLS> and <SEP>) and tokens to
# the sequence, as well as compute the attention masks.
paraphrase = tokenizer(sequence_0, sequence_2, return_tensors="pt") # Changed to return_tensors="pt"
not_paraphrase = tokenizer(sequence_0, sequence_1, return_tensors="pt") # Changed to return_tensors="pt"

paraphrase_classification_logits = model(**paraphrase).logits
not_paraphrase_classification_logits = model(**not_paraphrase).logits

paraphrase_results = torch.softmax(paraphrase_classification_logits, dim=1).tolist()[0] # Changed to torch.softmax
not_paraphrase_results = torch.softmax(not_paraphrase_classification_logits, dim=1).tolist()[0] # Changed to torch.softmax

# Should be paraphrase
for i in range(len(classes)):
    print(f"{classes[i]}: {int(round(paraphrase_results[i] * 100))}%")
```

```
not paraphrase: 10%
is paraphrase: 90%
```

```
# Should not be paraphrase
for i in range(len(classes)):
    print(f"{classes[i]}: {int(round(not_paraphrase_results[i] * 100))}%")
```

```
not paraphrase: 94%
is paraphrase: 6%
```

✓ Extractive Question Answering

Extractive Question Answering is the task of extracting an answer from a text given a question. An example of a question answering dataset is the SQuAD dataset, which is entirely based on that task. If you would like to fine-tune a model on a SQuAD task, you may leverage the [run_qa.py](#) and [run_tf_squad.py](#) scripts.

Here is an example of using pipelines to do question answering: extracting an answer from a text given a question. It leverages a fine-tuned model on SQuAD.

```
from transformers import pipeline

question_answerer = pipeline("question-answering")

context = r"""
Extractive Question Answering is the task of extracting an answer from a text given a question. An example of a
question answering dataset is the SQuAD dataset, which is entirely based on that task. If you would like to fine-tune
a model on a SQuAD task, you may leverage the examples/pytorch/question-answering/run_squad.py script.
"""

No model was supplied, defaulted to distilbert/distilbert-base-cased-distilled-squad and revision 564e9b5 (https://huggingface
Using a pipeline without specifying a model name and revision in production is not recommended.
config.json: 100% 473/473 [00:00<00:00, 17.3kB/s]

model.safetensors: 100% 261M/261M [00:03<00:00, 81.8MB/s]

tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 2.07kB/s]

vocab.txt: 213k/? [00:00<00:00, 10.0MB/s]

tokenizer.json: 436k/? [00:00<00:00, 8.03MB/s]

Fetching 0 files: 0/0 [00:00<?, ?it/s]

Fetching 1 files: 100% 1/1 [00:00<00:00, 33.10it/s]

Fetching 0 files: 0/0 [00:00<?, ?it/s]

Device set to use cuda:0
```

This returns an answer extracted from the text, a confidence score, alongside "start" and "end" values, which are the positions of the extracted answer in the text.

```
result = question_answerer(question="What is extractive question answering?", context=context)
print(
    f"Answer: '{result['answer']}', score: {round(result['score'], 4)}, start: {result['start']}, end: {result['end']}"
)
```

```
Answer: 'the task of extracting an answer from a text given a question', score: 0.6177, start: 34, end: 95
```

```
result = question_answerer(question="What is a good example of a question answering dataset?", context=context)
print(
    f"Answer: '{result['answer']}', score: {round(result['score'], 4)}, start: {result['start']}, end: {result['end']}"
)
```

```
Answer: 'SQuAD dataset', score: 0.5153, start: 147, end: 160
```

Here is an example of question answering using a model and a tokenizer. The process is the following:

1. Instantiate a tokenizer and a model from the checkpoint name. The model is identified as a BERT model and loads it with the weights stored in the checkpoint.
2. Define a text and a few questions.
3. Iterate over the questions and build a sequence from the text and the current question, with the correct model-specific separators, token type ids and attention masks.

4. Pass this sequence through the model. This outputs a range of scores across the entire sequence tokens (question and text), for both the start and end positions.
5. Compute the softmax of the result to get probabilities over the tokens.
6. Fetch the tokens from the identified start and stop values, convert those tokens to a string.
7. Print the results.

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

tokenizer = AutoTokenizer.from_pretrained("bert-large-uncased-whole-word-masking-finetuned-squad")
model = AutoModelForQuestionAnswering.from_pretrained("bert-large-uncased-whole-word-masking-finetuned-squad")

text = r"""
🤗 Transformers (formerly known as pytorch-transformers and pytorch-pretrained-bert) provides general-purpose
architectures (BERT, GPT-2, RoBERTa, XLM, DistilBert, XLNet...) for Natural Language Understanding (NLU) and Natural
Language Generation (NLG) with over 32+ pretrained models in 100+ languages and deep interoperability between
TensorFlow 2.0 and PyTorch.
"""

questions = [
    "How many pretrained models are available in 🤗 Transformers?",
    "What does 🤗 Transformers provide?",
    "🤗 Transformers provides interoperability between which frameworks?",
]

for question in questions:
    inputs = tokenizer(question, text, add_special_tokens=True, return_tensors="pt")
    input_ids = inputs["input_ids"].tolist()[0]

    outputs = model(**inputs)
    answer_start_scores = outputs.start_logits
    answer_end_scores = outputs.end_logits

    # Get the most likely beginning of answer with the argmax of the score
    answer_start = torch.argmax(answer_start_scores)
    # Get the most likely end of answer with the argmax of the score
    answer_end = torch.argmax(answer_end_scores) + 1

    answer = tokenizer.convert_tokens_to_string(
        tokenizer.convert_ids_to_tokens(input_ids[answer_start:answer_end])
    )

    print(f"Question: {question}")
    print(f"Answer: {answer}")
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.60kB/s]
config.json: 100% 443/443 [00:00<00:00, 21.3kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.82MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 3.42MB/s]
model.safetensors: 100% 1.34G/1.34G [00:18<00:00, 132MB/s]
```

Some weights of the model checkpoint at bert-large-uncased-whole-word-masking-finetuned-squad were not used when initializing E
- This IS expected if you are initializing BertForQuestionAnswering from the checkpoint of a model trained on another task or v
- This IS NOT expected if you are initializing BertForQuestionAnswering from the checkpoint of a model that you expect to be ex
Question: How many pretrained models are available in 🤗 Transformers?
Answer: over 32 +
Question: What does 🤗 Transformers provide?
Answer: general - purpose architectures
Question: 🤗 Transformers provides interoperability between which frameworks?
Answer: tensorflow 2. 0 and pytorch

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering # Changed to AutoModelForQuestionAnswering
import torch # Added torch import

tokenizer = AutoTokenizer.from_pretrained("bert-large-uncased-whole-word-masking-finetuned-squad")
model = AutoModelForQuestionAnswering.from_pretrained("bert-large-uncased-whole-word-masking-finetuned-squad") # Changed to Ai

text = r"""
🤗 Transformers (formerly known as pytorch-transformers and pytorch-pretrained-bert) provides general-purpose
architectures (BERT, GPT-2, RoBERTa, XLM, DistilBert, XLNet...) for Natural Language Understanding (NLU) and Natural
Language Generation (NLG) with over 32+ pretrained models in 100+ languages and deep interoperability between
TensorFlow 2.0 and PyTorch.
"""

questions = [
    "How many pretrained models are available in 🤗 Transformers?",
    "What does 🤗 Transformers provide?",
```

```

]

" 🤗 Transformers provides interoperability between which frameworks?",

]

for question in questions:
    inputs = tokenizer(question, text, add_special_tokens=True, return_tensors="pt") # Changed return_tensors to "pt"
    input_ids = inputs["input_ids"].tolist()[0] # Changed tolist()[0]

    outputs = model(**inputs)
    answer_start_scores = outputs.start_logits
    answer_end_scores = outputs.end_logits

    # Get the most likely beginning of answer with the argmax of the score
    answer_start = torch.argmax(answer_start_scores) # Changed to torch.argmax
    # Get the most likely end of answer with the argmax of the score
    answer_end = torch.argmax(answer_end_scores) + 1 # Changed to torch.argmax + 1

    answer = tokenizer.convert_tokens_to_string(
        tokenizer.convert_ids_to_tokens(input_ids[answer_start:answer_end])
    )

    print(f"Question: {question}")
    print(f"Answer: {answer}")

```

Some weights of the model checkpoint at bert-large-uncased-whole-word-masking-finetuned-squad were not used when initializing B

- This IS expected if you are initializing BertForQuestionAnswering from the checkpoint of a model trained on another task or v
- This IS NOT expected if you are initializing BertForQuestionAnswering from the checkpoint of a model that you expect to be ex

Question: How many pretrained models are available in 🤗 Transformers?
 Answer: over 32 +

Question: What does 🤗 Transformers provide?
 Answer: general - purpose architectures

Question: 🤗 Transformers provides interoperability between which frameworks?
 Answer: tensorflow 2. 0 and pytorch

✓ Language Modeling

Language modeling is the task of fitting a model to a corpus, which can be domain specific. All popular transformer-based models are trained using a variant of language modeling, e.g. BERT with masked language modeling, GPT-2 with causal language modeling.

Language modeling can be useful outside of pretraining as well, for example to shift the model distribution to be domain-specific: using a language model trained over a very large corpus, and then fine-tuning it to a news dataset or on scientific papers e.g. [LysandreJik/arxiv-nlp](#).

✓ Masked Language Modeling

Masked language modeling is the task of masking tokens in a sequence with a masking token, and prompting the model to fill that mask with an appropriate token. This allows the model to attend to both the right context (tokens on the right of the mask) and the left context (tokens on the left of the mask). Such a training creates a strong basis for downstream tasks requiring bi-directional context, such as SQuAD (question answering, see [Lewis, Luj, Goyal et al.](#), part 4.2). If you would like to fine-tune a model on a masked language modeling task, you may leverage the [run_mlm.py](#) script.

Here is an example of using pipelines to replace a mask from a sequence:

```

from transformers import pipeline

unmasker = pipeline("fill-mask")

```

No model was supplied, defaulted to distilbert/distilroberta-base and revision fb53ab8 (<https://huggingface.co/distilbert/distilroberta-base>). Using a pipeline without specifying a model name and revision in production is not recommended.

config.json: 100% 480/480 [00:00<00:00, 24.8kB/s]

model.safetensors: 100% 331M/331M [00:05<00:00, 86.2MB/s]

Some weights of the model checkpoint at distilbert/distilroberta-base were not used when initializing RobertaForMaskedLM: ['rot

- This IS expected if you are initializing RobertaForMaskedLM from the checkpoint of a model trained on another task or with ar
- This IS NOT expected if you are initializing RobertaForMaskedLM from the checkpoint of a model that you expect to be exactly

tokenizer_config.json: 100% 25.0/25.0 [00:00<00:00, 1.43kB/s]

vocab.json: 899k/? [00:00<00:00, 29.0MB/s]

merges.txt: 456k/? [00:00<00:00, 13.2MB/s]

tokenizer.json: 1.36M/? [00:00<00:00, 18.5MB/s]

Device set to use cuda:0

This outputs the sequences with the mask filled, the confidence score, and the token id in the tokenizer vocabulary:

```
from pprint import pprint

pprint(
    unmasker(
        f"HuggingFace is creating a {tokenizer.mask_token} that the community uses to solve NLP tasks."
    )
)

[{'score': 0.1792750507593155,
  'sequence': 'HuggingFace is creating a tool that the community uses to solve '
              'NLP tasks.',
  'token': 3944,
  'token_str': ' tool'},
 {'score': 0.11349441111087799,
  'sequence': 'HuggingFace is creating a framework that the community uses to '
              'solve NLP tasks.',
  'token': 7208,
  'token_str': ' framework'},
 {'score': 0.052435584366321564,
  'sequence': 'HuggingFace is creating a library that the community uses to '
              'solve NLP tasks.',
  'token': 5560,
  'token_str': ' library'},
 {'score': 0.03493529185652733,
  'sequence': 'HuggingFace is creating a database that the community uses to '
              'solve NLP tasks.',
  'token': 8503,
  'token_str': ' database'},
 {'score': 0.028602521866559982,
  'sequence': 'HuggingFace is creating a prototype that the community uses to '
              'solve NLP tasks.',
  'token': 17715,
  'token_str': ' prototype'}]
```

Here is an example of doing masked language modeling using a model and a tokenizer. The process is the following:

1. Instantiate a tokenizer and a model from the checkpoint name. The model is identified as a DistilBERT model and loads it with the weights stored in the checkpoint.
2. Define a sequence with a masked token, placing the `tokenizer.mask_token` instead of a word.
3. Encode that sequence into a list of IDs and find the position of the masked token in that list.
4. Retrieve the predictions at the index of the mask token: this tensor has the same size as the vocabulary, and the values are the scores attributed to each token. The model gives higher score to tokens it deems probable in that context.
5. Retrieve the top 5 tokens using the PyTorch `topk` or TensorFlow `top_k` methods.
6. Replace the mask token by the tokens and print the results

```
from transformers import AutoModelForMaskedLM, AutoTokenizer
import torch

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-cased")
model = AutoModelForMaskedLM.from_pretrained("distilbert-base-cased")

sequence = (
    "Distilled models are smaller than the models they mimic. Using them instead of the large "
    f"versions would help {tokenizer.mask_token} our carbon footprint."
)

inputs = tokenizer(sequence, return_tensors="pt")
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1]

token_logits = model(**inputs).logits
mask_token_logits = token_logits[0, mask_token_index, :]

top_5_tokens = torch.topk(mask_token_logits, 5, dim=1).indices[0].tolist()

for token in top_5_tokens:
    print(sequence.replace(tokenizer.mask_token, tokenizer.decode([token])))
```

```
tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 3.15kB/s]
config.json: 100% 465/465 [00:00<00:00, 51.1kB/s]
vocab.txt: 100% 213k/213k [00:00<00:00, 2.88MB/s]
tokenizer.json: 100% 436k/436k [00:00<00:00, 12.5MB/s]
model.safetensors: 100% 263M/263M [00:13<00:00, 15.0MB/s]
```

Distilled models are smaller than the models they mimic. Using them instead of the large versions would help reduce our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help increase our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help decrease our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help offset our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help improve our carbon footprint.

```
from transformers import AutoModelForMaskedLM, AutoTokenizer # Changed to AutoModelForMaskedLM
import torch # Changed to import torch

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-cased")
model = AutoModelForMaskedLM.from_pretrained("distilbert-base-cased")

sequence = (
    "Distilled models are smaller than the models they mimic. Using them instead of the large "
    f"versions would help {tokenizer.mask_token} our carbon footprint."
)

inputs = tokenizer(sequence, return_tensors="pt") # Changed return_tensors to "pt"
mask_token_index = torch.where(inputs["input_ids"] == tokenizer.mask_token_id)[1] # Changed to torch.where

token_logits = model(**inputs).logits
mask_token_logits = token_logits[0, mask_token_index, :]

top_5_tokens = torch.topk(mask_token_logits, 5, dim=1).indices[0].tolist() # Changed to torch.topk

for token in top_5_tokens:
    print(sequence.replace(tokenizer.mask_token, tokenizer.decode([token])))
```

Distilled models are smaller than the models they mimic. Using them instead of the large versions would help reduce our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help increase our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help decrease our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help offset our carbon footprint.
 Distilled models are smaller than the models they mimic. Using them instead of the large versions would help improve our carbon footprint.

This prints five sequences, with the top 5 tokens predicted by the model.

▼ Causal Language Modeling

Causal language modeling is the task of predicting the token following a sequence of tokens. In this situation, the model only attends to the left context (tokens on the left of the mask). Such a training is particularly interesting for generation tasks. If you would like to fine-tune a model on a causal language modeling task, you may leverage the [run_clm.py](#) script.

Usually, the next token is predicted by sampling from the logits of the last hidden state the model produces from the input sequence.

Here is an example of using the tokenizer and model and leveraging the [top_k_top_p_filtering\(\)](#) method to sample the next token following an input sequence of tokens.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

tokenizer = AutoTokenizer.from_pretrained("gpt2") # Changed model identifier
model = AutoModelForCausalLM.from_pretrained("gpt2") # Changed model identifier

sequence = f"Hugging Face is based in DUMBO, New York City, and"

inputs = tokenizer(sequence, return_tensors="pt")
input_ids = inputs["input_ids"]

# Set pad_token_id
if tokenizer.pad_token_id is None:
    tokenizer.pad_token_id = tokenizer.eos_token_id

# Use model.generate() for text generation with sampling
outputs = model.generate(
    input_ids,
    attention_mask=inputs["attention_mask"], # Pass attention mask
    max_length=len(input_ids[0]) + 20,
    num_return_sequences=1,
    do_sample=True,
```

```

        top_k=50,
        top_p=1.0,
        pad_token_id=tokenizer.pad_token_id # Pass pad_token_id
    )

```

```

resulting_string = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(resulting_string)

```

```

tokenizer_config.json: 100%          26.0/26.0 [00:00<00:00, 3.01kB/s]
config.json: 100%                   665/665 [00:00<00:00, 66.3kB/s]
vocab.json: 100%                    1.04M/1.04M [00:00<00:00, 4.83MB/s]
merges.txt: 100%                    456k/456k [00:00<00:00, 3.72MB/s]
tokenizer.json: 100%                1.36M/1.36M [00:00<00:00, 10.2MB/s]
model.safetensors: 100%             548M/548M [00:11<00:00, 60.4MB/s]
generation_config.json: 100%        124/124 [00:00<00:00, 10.5kB/s]
Hugging Face is based in DUMBO, New York City, and takes place in the "Sugar Mill" a year after the death of a pregnant mother

```

Here is an example of using the tokenizer and model and leveraging the [tf_top_k_top_p_filtering\(\)](#) method to sample the next token following an input sequence of tokens.

```

from transformers import AutoModelForCausalLM, AutoTokenizer

# Load tokenizer and model
tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

# Input text
sequence = "Hugging Face is based in DUMBO, New York City, and"

# Tokenize (PyTorch tensors)
inputs = tokenizer(sequence, return_tensors="pt")

# Generate text
outputs = model.generate(
    inputs["input_ids"],
    attention_mask=inputs["attention_mask"],
    max_length=inputs["input_ids"].shape[1] + 20,
    num_return_sequences=1,
    do_sample=True,
    top_k=50,
    top_p=1.0,
    pad_token_id=tokenizer.eos_token_id # GPT-2 doesn't have pad_token, use eos_token_id
)

# Decode back to string
resulting_string = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(resulting_string)

```

```

Hugging Face is based in DUMBO, New York City, and features exclusive content, exclusive art, exclusive content, and exclusive

```

This outputs a (hopefully) coherent next token following the original sequence, which in our case is the word *is* or *features*.

In the next section, we show how [generation_utils.GenerationMixin.generate\(\)](#) can be used to generate multiple tokens up to a specified length instead of one token at a time.

✓ Text Generation

In text generation (*a.k.a open-ended text generation*) the goal is to create a coherent portion of text that is a continuation from the given context. The following example shows how *GPT-2* can be used in pipelines to generate text. As a default all models apply *Top-K* sampling when used in pipelines, as configured in their respective configurations (see [gpt-2 config](#) for example).

```

from transformers import pipeline

text_generator = pipeline("text-generation")
print(text_generator("As far as I am concerned, I will", max_length=50, do_sample=False))

```

No model was supplied, defaulted to openai-community/gpt2 and revision 607a30d (<https://huggingface.co/openai-community/gpt2>). Using a pipeline without specifying a model name and revision in production is not recommended.

```
config.json: 100%                               665/665 [00:00<00:00, 54.3kB/s]

model.safetensors: 100%                         548M/548M [00:01<00:00, 34.9MB/s]

generation_config.json: 100%                    124/124 [00:00<00:00, 13.8kB/s]

tokenizer_config.json: 100%                     26.0/26.0 [00:00<00:00, 2.80kB/s]

vocab.json: 1.04M/? [00:00<00:00, 46.2MB/s]

merges.txt: 456k/? [00:00<00:00, 24.2MB/s]

tokenizer.json: 1.36M/? [00:00<00:00, 53.3MB/s]
```

Device set to use cuda:0

Truncation was not explicitly activated but `max\_length` is provided a specific value, please use `truncation=True` to explicit. The following generation flags are not valid and may be ignored: ['temperature']. Set `TRANSFORMERS\_VERBOSITY=info` for more details. Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation. Both `max\_new\_tokens` (=256) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the docs for more details. [0] generated_text: 'As far as I am concerned, I will be the first to admit that I am not a fan of the idea of a "free market."

Here, the model generates a random text with a total maximal length of 50 tokens from context *"As far as I am concerned, I will".* Behind the scenes, the pipeline object calls the method [PreTrainedModel.generate\(\)](#) to generate text. The default arguments for this method can be overridden in the pipeline, as is shown above for the arguments `max_length` and `do_sample`.

Below is an example of text generation using `XLNet` and its tokenizer, which includes calling `generate()` directly:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("xlnet-base-cased")
tokenizer = AutoTokenizer.from_pretrained("xlnet-base-cased")

# Padding text helps XLNet with short prompts - proposed by Aman Rusia in https://github.com/rusiaaman/XLNet-gen#methodology
PADDING_TEXT = """In 1991, the remains of Russian Tsar Nicholas II and his family
(except for Alexei and Maria) are discovered.
The voice of Nicholas's young son, Tsarevich Alexei Nikolaevich, narrates the
remainder of the story. 1883 Western Siberia,
a young Grigori Rasputin is asked by his father and a group of men to perform magic.
Rasputin has a vision and denounces one of the men as a horse thief. Although his
father initially slaps him for making such an accusation, Rasputin watches as the
man is chased outside and beaten. Twenty years later, Rasputin sees a vision of
the Virgin Mary, prompting him to become a priest. Rasputin quickly becomes famous,
with people, even a bishop, begging for his blessing. <eos> </s> <eos>"""

prompt = "Today the weather is really nice and I am planning on "
inputs = tokenizer(PADDING_TEXT + prompt, add_special_tokens=False, return_tensors="pt")["input_ids"]

prompt_length = len(tokenizer.decode(inputs[0]))
outputs = model.generate(inputs, max_length=250, do_sample=True, top_p=0.95, top_k=60)
generated = prompt + tokenizer.decode(outputs[0])[prompt_length + 1 :]

print(generated)
```

```
config.json: 100%                               760/760 [00:00<00:00, 64.3kB/s]

pytorch_model.bin: 100%                        467M/467M [00:11<00:00, 32.3MB/s]

generation_config.json: 100%                   137/137 [00:00<00:00, 9.48kB/s]

model.safetensors: 100%                       467M/467M [00:05<00:00, 116MB/s]

spiece.model: 100%                             798k/798k [00:00<00:00, 4.25MB/s]

tokenizer.json: 100%                           1.38M/1.38M [00:00<00:00, 5.39MB/s]
```

This is a friendly reminder - the current text generation call will exceed the model's predefined maximum length (-1). Depending on the prompt, the generated text may be truncated. Today the weather is really nice and I am planning on riding the bike to see my new local friend, Rob. This is how I feel about

```
from transformers import TFAutoModelForCausalLM, AutoTokenizer

model = TFAutoModelForCausalLM.from_pretrained("xlnet-base-cased")
tokenizer = AutoTokenizer.from_pretrained("xlnet-base-cased")

# Padding text helps XLNet with short prompts - proposed by Aman Rusia in https://github.com/rusiaaman/XLNet-gen#methodology
PADDING_TEXT = """In 1991, the remains of Russian Tsar Nicholas II and his family
(except for Alexei and Maria) are discovered.
The voice of Nicholas's young son, Tsarevich Alexei Nikolaevich, narrates the
remainder of the story. 1883 Western Siberia,
a young Grigori Rasputin is asked by his father and a group of men to perform magic.
Rasputin has a vision and denounces one of the men as a horse thief. Although his
```

```
father initially slaps him for making such an accusation, Rasputin watches as the
man is chased outside and beaten. Twenty years later, Rasputin sees a vision of
the Virgin Mary, prompting him to become a priest. Rasputin quickly becomes famous,
with people, even a bishop, begging for his blessing. <eod> </s> <eos>"""
```

```
prompt = "Today the weather is really nice and I am planning on "
inputs = tokenizer(PADDING_TEXT + prompt, add_special_tokens=False, return_tensors="tf")["input_ids"]
```

```
prompt_length = len(tokenizer.decode(inputs[0]))
outputs = model.generate(inputs, max_length=250, do_sample=True, top_p=0.95, top_k=60)
generated = prompt + tokenizer.decode(outputs[0])[prompt_length + 1 :]
```

```
print(generated)
```

```
tf_model.h5: 100%
```

```
565M/565M [00:16<00:00, 32.4MB/s]
```

```
TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or
/usr/local/lib/python3.12/dist-packages/tf_keras/src/initializers/initializers.py:121: UserWarning: The initializer TruncatedNormal
warnings.warn(
All model checkpoint layers were used when initializing TFXLNetLMHeadModel.
```

```
All the layers of TFXLNetLMHeadModel were initialized from the model checkpoint at xlnet-base-cased.
If your task is similar to the task the model of the checkpoint was trained on, you can already use TFXLNetLMHeadModel for pre-training.
TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or
Today the weather is really nice and I am planning on having a lunch with our family over the next few hours. This is good news
```

Text generation is currently possible with *GPT-2*, *OpenAi-GPT*, *CTRL*, *XLNet*, *Transfo-XL* and *Reformer* in PyTorch and for most models in Tensorflow as well. As can be seen in the example above *XLNet* and *Transfo-XL* often need to be padded to work well. GPT-2 is usually a good choice for *open-ended text generation* because it was trained on millions of webpages with a causal language modeling objective.

For more information on how to apply different decoding strategies for text generation, please also refer to our text generation blog post [here](#).

▼ Named Entity Recognition

Named Entity Recognition (NER) is the task of classifying tokens according to a class, for example, identifying a token as a person, an organisation or a location. An example of a named entity recognition dataset is the CoNLL-2003 dataset, which is entirely based on that task. If you would like to fine-tune a model on an NER task, you may leverage the [run_ner.py](#) script.

Here is an example of using pipelines to do named entity recognition, specifically, trying to identify tokens as belonging to one of 9 classes:

- O, Outside of a named entity
- B-MIS, Beginning of a miscellaneous entity right after another miscellaneous entity
- I-MIS, Miscellaneous entity
- B-PER, Beginning of a person's name right after another person's name
- I-PER, Person's name
- B-ORG, Beginning of an organisation right after another organisation
- I-ORG, Organisation
- B-LOC, Beginning of a location right after another location
- I-LOC, Location

It leverages a fine-tuned model on CoNLL-2003, fine-tuned by [@stefan-it](#) from [dbmdz](#).

```
from transformers import pipeline
```

```
ner_pipe = pipeline("ner")
```

```
sequence = ""Hugging Face Inc. is a company based in New York City. Its headquarters are in DUMBO,
therefore very close to the Manhattan Bridge which is visible from the window.""
```

```
No model was supplied, defaulted to dbmdz/bert-large-cased-finetuned-conll03-english and revision 4c53496 (https://huggingface.co/dbmdz/bert-large-cased-finetuned-conll03-english)
Using a pipeline without specifying a model name and revision is not recommended.
```

```
config.json: 100%
```

```
998/998 [00:00<00:00, 116kB/s]
```

```
model.safetensors: 100%
```

```
1.33G/1.33G [01:09<00:00, 27.6MB/s]
```

```
Some weights of the model checkpoint at dbmdz/bert-large-cased-finetuned-conll03-english were not used when initializing BertForTokenClassification
- This IS expected if you are initializing BertForTokenClassification from the checkpoint of a model trained on another task or
- This IS NOT expected if you are initializing BertForTokenClassification from the checkpoint of a model that you expect to be initialized with those weights
```

```
tokenizer_config.json: 100%
```

```
60.0/60.0 [00:00<00:00, 7.01kB/s]
```

```
vocab.txt: 213k/? [00:00<00:00, 16.3MB/s]
```

```
Device set to use cuda:0
```

This outputs a list of all words that have been identified as one of the entities from the 9 classes defined above. Here are the expected results:

```
for entity in ner_pipe(sequence):
    print(entity)

{'entity': 'I-ORG', 'score': np.float32(0.99957865), 'index': 1, 'word': 'Hu', 'start': 0, 'end': 2}
{'entity': 'I-ORG', 'score': np.float32(0.9909764), 'index': 2, 'word': '##gging', 'start': 2, 'end': 7}
{'entity': 'I-ORG', 'score': np.float32(0.9982224), 'index': 3, 'word': 'Face', 'start': 8, 'end': 12}
{'entity': 'I-ORG', 'score': np.float32(0.9994879), 'index': 4, 'word': 'Inc', 'start': 13, 'end': 16}
{'entity': 'I-LOC', 'score': np.float32(0.9994344), 'index': 11, 'word': 'New', 'start': 40, 'end': 43}
{'entity': 'I-LOC', 'score': np.float32(0.99931955), 'index': 12, 'word': 'York', 'start': 44, 'end': 48}
{'entity': 'I-LOC', 'score': np.float32(0.9993794), 'index': 13, 'word': 'City', 'start': 49, 'end': 53}
{'entity': 'I-LOC', 'score': np.float32(0.98625827), 'index': 19, 'word': 'D', 'start': 79, 'end': 80}
{'entity': 'I-LOC', 'score': np.float32(0.951427), 'index': 20, 'word': '##UM', 'start': 80, 'end': 82}
{'entity': 'I-LOC', 'score': np.float32(0.93365926), 'index': 21, 'word': '##BO', 'start': 82, 'end': 84}
{'entity': 'I-LOC', 'score': np.float32(0.9761654), 'index': 28, 'word': 'Manhattan', 'start': 114, 'end': 123}
{'entity': 'I-LOC', 'score': np.float32(0.9914629), 'index': 29, 'word': 'Bridge', 'start': 124, 'end': 130}
```

Note how the tokens of the sequence "Hugging Face" have been identified as an organisation, and "New York City", "DUMBO" and "Manhattan Bridge" have been identified as locations.

Here is an example of doing named entity recognition, using a model and a tokenizer. The process is the following:

1. Instantiate a tokenizer and a model from the checkpoint name. The model is identified as a BERT model and loads it with the weights stored in the checkpoint.
2. Define a sequence with known entities, such as "Hugging Face" as an organisation and "New York City" as a location.
3. Split words into tokens so that they can be mapped to predictions. We use a small hack by, first, completely encoding and decoding the sequence, so that we're left with a string that contains the special tokens.
4. Encode that sequence into IDs (special tokens are added automatically).
5. Retrieve the predictions by passing the input to the model and getting the first output. This results in a distribution over the 9 possible classes for each token. We take the argmax to retrieve the most likely class for each token.
6. Zip together each token with its prediction and print it.

```
from transformers import AutoModelForTokenClassification, AutoTokenizer
import torch

model = AutoModelForTokenClassification.from_pretrained("dbmdz/bert-large-cased-finetuned-conll103-english")
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

sequence = (
    "Hugging Face Inc. is a company based in New York City. Its headquarters are in DUMBO, "
    "therefore very close to the Manhattan Bridge."
)

inputs = tokenizer(sequence, return_tensors="pt")
tokens = inputs.tokens()

outputs = model(**inputs).logits
predictions = torch.argmax(outputs, dim=2)
```

Some weights of the model checkpoint at dbmdz/bert-large-cased-finetuned-conll103-english were not used when initializing BertForTokenClassification from the checkpoint of a model trained on another task or - This IS NOT expected if you are initializing BertForTokenClassification from the checkpoint of a model that you expect to be

tokenizer_config.json: 100%	49.0/49.0 [00:00<00:00, 4.16kB/s]
config.json: 100%	570/570 [00:00<00:00, 55.9kB/s]
vocab.txt: 100%	213k/213k [00:00<00:00, 3.21MB/s]
tokenizer.json: 100%	436k/436k [00:00<00:00, 6.37MB/s]

```
from transformers import AutoModelForTokenClassification, AutoTokenizer # Changed to AutoModelForTokenClassification
import torch

model = AutoModelForTokenClassification.from_pretrained("dbmdz/bert-large-cased-finetuned-conll103-english")
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

sequence = (
    "Hugging Face Inc. is a company based in New York City. Its headquarters are in DUMBO, "
    "therefore very close to the Manhattan Bridge."
)

inputs = tokenizer(sequence, return_tensors="pt")
tokens = inputs.tokens()
```

```
outputs = model(**inputs).logits
predictions = torch.argmax(outputs, dim=2)
```

Some weights of the model checkpoint at dbmdz/bert-large-cased-finetuned-conll03-english were not used when initializing BertForTokenClassification from the checkpoint of a model trained on another task or

- This IS expected if you are initializing BertForTokenClassification from the checkpoint of a model trained on another task or
- This IS NOT expected if you are initializing BertForTokenClassification from the checkpoint of a model that you expect to be

This outputs a list of each token mapped to its corresponding prediction. Differently from the pipeline, here every token has a prediction as we didn't remove the "O"th class, which means that no particular entity was found on that token.

In the above example, `predictions` is an integer that corresponds to the predicted class. We can use the `model.config.id2label` property in order to recover the class name corresponding to the class number, which is illustrated below:

```
for token, prediction in zip(tokens, predictions[0].numpy()):
    # Check if the predicted ID is in the id2label mapping
    if prediction in model.config.id2label:
        print((token, model.config.id2label[prediction]))
    else:
        print((token, f"Unknown Label ID: {prediction}"))
```

```
('[CLS]', 'O')
('Hu', 'I-ORG')
('##gging', 'I-ORG')
('Face', 'I-ORG')
('Inc', 'I-ORG')
('.', 'O')
('is', 'O')
('a', 'O')
('company', 'O')
('based', 'O')
('in', 'O')
('New', 'I-LOC')
('York', 'I-LOC')
('City', 'I-LOC')
('.', 'O')
('Its', 'O')
('headquarters', 'O')
('are', 'O')
('in', 'O')
('D', 'I-LOC')
('##UM', 'I-LOC')
('##BO', 'I-LOC')
(',', 'O')
('therefore', 'O')
('very', 'O')
('close', 'O')
('to', 'O')
('the', 'O')
('Manhattan', 'I-LOC')
('Bridge', 'I-LOC')
('.', 'O')
('[SEP]', 'O')
```

✓ Summarization

Summarization is the task of summarizing a document or an article into a shorter text. If you would like to fine-tune a model on a summarization task, you may leverage the [run_summarization.py](#) script.

An example of a summarization dataset is the CNN / Daily Mail dataset, which consists of long news articles and was created for the task of summarization. If you would like to fine-tune a model on a summarization task, various approaches are described in this [document](#).

Here is an example of using the pipelines to do summarization. It leverages a Bart model that was fine-tuned on the CNN / Daily Mail data set.

```
from transformers import pipeline

summarizer = pipeline("summarization")
```

```
ARTICLE = """ New York (CNN)When Liana Barrientos was 23 years old, she got married in Westchester County, New York.
A year later, she got married again in Westchester County, but to a different man and without divorcing her first husband.
Only 18 days after that marriage, she got hitched yet again. Then, Barrientos declared "I do" five more times, sometimes only
In 2010, she married once more, this time in the Bronx. In an application for a marriage license, she stated it was her "first
Barrientos, now 39, is facing two criminal counts of "offering a false instrument for filing in the first degree," referring to
2010 marriage license application, according to court documents.
Prosecutors said the marriages were part of an immigration scam.
On Friday, she pleaded not guilty at State Supreme Court in the Bronx, according to her attorney, Christopher Wright, who dec
After leaving court, Barrientos was arrested and charged with theft of service and criminal trespass for allegedly sneaking in
Annette Markowski, a police spokeswoman. In total, Barrientos has been married 10 times, with nine of her marriages occurring
All occurred either in Westchester County, Long Island, New Jersey or the Bronx. She is believed to still be married to four i
```

Prosecutors said the immigration scam involved some of her husbands, who filed for permanent residence status shortly after the divorces. Any divorces happened only after such filings were approved. It was unclear whether any of the men will be prosecuted. The case was referred to the Bronx District Attorney's Office by Immigration and Customs Enforcement and the Department of Homeland Security Investigation Division. Seven of the men are from so-called "red-flagged" countries, including Egypt, Turkey, Georgia, Pakistan. Her eighth husband, Rashid Rajput, was deported in 2006 to his native Pakistan after an investigation by the Joint Terrorism Task Force. If convicted, Barrientos faces up to four years in prison. Her next court appearance is scheduled for May 18.

No model was supplied, defaulted to sshleifer/distilbart-cnn-12-6 and revision a4f8f3e (<https://huggingface.co/sshleifer/distilbart-cnn-12-6>)
Using a pipeline without specifying a model name and revision in production is not recommended.

config.json: 1.80k/? [00:00<00:00, 110kB/s]

pytorch_model.bin: 100% 1.22G/1.22G [01:13<00:00, 15.7MB/s]

model.safetensors: 100% 1.22G/1.22G [00:42<00:00, 13.0MB/s]

tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 678B/s]

vocab.json: 899k/? [00:00<00:00, 21.9MB/s]

merges.txt: 456k/? [00:00<00:00, 10.4MB/s]

Device set to use cuda:0

Because the summarization pipeline depends on the `PreTrainedModel.generate()` method, we can override the default arguments of `PreTrainedModel.generate()` directly in the pipeline for `max_length` and `min_length` as shown below. This outputs the following summary:

```
from transformers import pipeline
```

```
summarizer = pipeline("summarization")
```

```
ARTICLE = """ New York (CNN)When Liana Barrientos was 23 years old, she got married in Westchester County, New York. A year later, she got married again in Westchester County, but to a different man and without divorcing her first husband. Only 18 days after that marriage, she got hitched yet again. Then, Barrientos declared "I do" five more times, sometimes only In 2010, she married once more, this time in the Bronx. In an application for a marriage license, she stated it was her "first marriage." Barrientos, now 39, is facing two criminal counts of "offering a false instrument for filing in the first degree," referring to her 2010 marriage license application, according to court documents. Prosecutors said the marriages were part of an immigration scam. On Friday, she pleaded not guilty at State Supreme Court in the Bronx, according to her attorney, Christopher Wright, who declined comment. After leaving court, Barrientos was arrested and charged with theft of service and criminal trespass for allegedly sneaking in to the home of Annette Markowski, a police spokeswoman. In total, Barrientos has been married 10 times, with nine of her marriages occurring between 2005 and 2010. All occurred either in Westchester County, Long Island, New Jersey or the Bronx. She is believed to still be married to four men, but has not been heard from in years. Prosecutors said the immigration scam involved some of her husbands, who filed for permanent residence status shortly after the marriages. Any divorces happened only after such filings were approved. It was unclear whether any of the men will be prosecuted. The case was referred to the Bronx District Attorney's Office by Immigration and Customs Enforcement and the Department of Homeland Security Investigation Division. Seven of the men are from so-called "red-flagged" countries, including Egypt, Turkey, Georgia, Pakistan and Venezuela. Her eighth husband, Rashid Rajput, was deported in 2006 to his native Pakistan after an investigation by the Joint Terrorism Task Force. If convicted, Barrientos faces up to four years in prison. Her next court appearance is scheduled for May 18.
```

```
print(summarizer(ARTICLE, max_length=130, min_length=30, do_sample=False))
```

No model was supplied, defaulted to sshleifer/distilbart-cnn-12-6 and revision a4f8f3e (<https://huggingface.co/sshleifer/distilbart-cnn-12-6>)
Using a pipeline without specifying a model name and revision in production is not recommended.

Device set to use cuda:0

```
[{'summary_text': ' Liana Barrientos, 39, is charged with two counts of "offering a false instrument for filing in the first degree" in connection with her immigration scam. She pleaded not guilty to the charges in court on Friday. Her next court appearance is scheduled for May 18.'}]
```

Here is an example of doing summarization using a model and a tokenizer. The process is the following:

1. Instantiate a tokenizer and a model from the checkpoint name. Summarization is usually done using an encoder-decoder model, such as `Bart` or `T5`.
2. Define the article that should be summarized.
3. Add the T5 specific prefix "summarize: ".
4. Use the `PreTrainedModel.generate()` method to generate the summary.

In this example we use Google's T5 model. Even though it was pre-trained only on a multi-task mixed dataset (including CNN / Daily Mail), it yields very good results.

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
```

```
model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")
```

```
tokenizer = AutoTokenizer.from_pretrained("t5-base")
```

```
# T5 uses a max_length of 512 so we cut the article to 512 tokens.
```

```
inputs = tokenizer("summarize: " + ARTICLE, return_tensors="pt", max_length=512, truncation=True)
```

```
outputs = model.generate(
    inputs["input_ids"], max_length=150, min_length=40, length_penalty=2.0, num_beams=4, early_stopping=True
)
```

```
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

```
config.json: 100% 1.21k/1.21k [00:00<00:00, 115kB/s]
model.safetensors: 100% 892M/892M [01:01<00:00, 26.3MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 17.0kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 5.72MB/s]
tokenizer.json: 100% 1.39M/1.39M [00:00<00:00, 19.2MB/s]
```

prosecutors say the marriages were part of an immigration scam . if convicted, barrientos faces two criminal counts of "offerir

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer # Changed to AutoModelForSeq2SeqLM
```

```
model = AutoModelForSeq2SeqLM.from_pretrained("t5-base") # Changed to AutoModelForSeq2SeqLM
tokenizer = AutoTokenizer.from_pretrained("t5-base")
```

```
# T5 uses a max_length of 512 so we cut the article to 512 tokens.
inputs = tokenizer("summarize: " + ARTICLE, return_tensors="pt", max_length=512, truncation=True) # Changed return_tensors to
outputs = model.generate(
    inputs["input_ids"], max_length=150, min_length=40, length_penalty=2.0, num_beams=4, early_stopping=True
)
```

```
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

prosecutors say the marriages were part of an immigration scam . if convicted, barrientos faces two criminal counts of "offerir

▼ Translation

Translation is the task of translating a text from one language to another. If you would like to fine-tune a model on a translation task, you may leverage the [run_translation.py](#) script.

An example of a translation dataset is the WMT English to German dataset, which has sentences in English as the input data and the corresponding sentences in German as the target data. If you would like to fine-tune a model on a translation task, various approaches are described in this [document](#).

Here is an example of using the pipelines to do translation. It leverages a T5 model that was only pre-trained on a multi-task mixture dataset (including WMT), yet, yielding impressive translation results.

```
from transformers import pipeline
```

```
translator = pipeline("translation_en_to_de")
print(translator("Hugging Face is a technology company based in New York and Paris", max_length=40))
```

No model was supplied, defaulted to google-t5/t5-base and revision a9723ea (<https://huggingface.co/google-t5/t5-base>). Using a pipeline without specifying a model name and revision in production is not recommended.

```
config.json: 1.21k/? [00:00<00:00, 89.2kB/s]
model.safetensors: 100% 892M/892M [00:09<00:00, 124MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 16.5kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 41.0MB/s]
tokenizer.json: 1.39M/? [00:00<00:00, 36.0MB/s]
```

```
Device set to use cuda:0
Both `max_new_tokens` (=256) and `max_length` (=40) seem to have been set. `max_new_tokens` will take precedence. Please refer t
[{'translation_text': 'Hugging Face ist ein Technologieunternehmen mit Sitz in New York und Paris.'}]
```

Because the translation pipeline depends on the `PreTrainedModel.generate()` method, we can override the default arguments of `PreTrainedModel.generate()` directly in the pipeline as is shown for `max_length` above.

Here is an example of doing translation using a model and a tokenizer. The process is the following:

1. Instantiate a tokenizer and a model from the checkpoint name. Summarization is usually done using an encoder-decoder model, such as `Bart` or `T5`.
2. Define the article that should be summarized.
3. Add the T5 specific prefix "translate English to German: "
4. Use the `PreTrainedModel.generate()` method to perform the translation.

```
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
```

```

model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")
tokenizer = AutoTokenizer.from_pretrained("t5-base")

inputs = tokenizer(
    "translate English to German: Hugging Face is a technology company based in New York and Paris",
    return_tensors="pt",
)
outputs = model.generate(inputs["input_ids"], max_length=40, num_beams=4, early_stopping=True)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

Hugging Face ist ein Technologieunternehmen mit Sitz in New York und Paris.

```

from transformers import AutoModelForSeq2SeqLM, AutoTokenizer # Changed to AutoModelForSeq2SeqLM
import torch # Added torch import

model = AutoModelForSeq2SeqLM.from_pretrained("t5-base") # Changed to AutoModelForSeq2SeqLM
tokenizer = AutoTokenizer.from_pretrained("t5-base")

inputs = tokenizer(
    "translate English to German: Hugging Face is a technology company based in New York and Paris",
    return_tensors="pt", # Changed to "pt"
)
outputs = model.generate(inputs["input_ids"], max_length=40, num_beams=4, early_stopping=True)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))

```

Hugging Face ist ein Technologieunternehmen mit Sitz in New York und Paris.

We get the same translation as with the pipeline example.

✓ Audio classification

Audio classification assigns a class to an audio signal. The Keyword Spotting dataset from the [SUPERB](#) benchmark is an example dataset that can be used for audio classification fine-tuning. This dataset contains ten classes of keywords for classification. If you'd like to fine-tune a model for audio classification, take a look at the [run_audio_classification.py](#) script or this [how-to guide](#).

The following examples demonstrate how to use a [pipeline\(\)](#) and a model and tokenizer for audio classification inference:

```
!pip install torchcodec
```

```

Collecting torchcodec
  Downloading torchcodec-0.7.0-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (9.4 kB)
  Downloading torchcodec-0.7.0-cp312-cp312-manylinux_2_28_x86_64.whl (1.4 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.4/1.4 MB 23.4 MB/s eta 0:00:00
Installing collected packages: torchcodec
Successfully installed torchcodec-0.7.0

```

```
!pip show torchcodec
```

```

Name: torchcodec
Version: 0.7.0
Summary: A video decoder for PyTorch
Home-page:
Author:
Author-email: PyTorch Team <packages@pytorch.org>
License:
Location: /usr/local/lib/python3.12/dist-packages
Requires:
Required-by:

```

```

from datasets import load_dataset, Audio
import torchaudio
from transformers import pipeline

# Load dataset with auto-decoding
dataset = load_dataset(
    "hf-internal-testing/librispeech_asr_demo",
    "clean",
    split="validation"
)
dataset = dataset.cast_column("audio", Audio(decode=True)) # Changed decode=False to decode=True

# Get the *full path*
audio_info = dataset[0]["audio"]
# Access the audio array and sampling rate directly from the decoded object

```

```

audio_array = audio_info["array"]
sampling_rate = audio_info["sampling_rate"]
print("Sampling rate:", sampling_rate)

# Classifier
audio_classifier = pipeline(
    task="audio-classification",
    model="ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition"
)

# Predict
predictions = audio_classifier({"array": audio_array, "sampling_rate": sampling_rate})
predictions = [{"score": round(pred["score"], 4), "label": pred["label"]} for pred in predictions]
print(predictions)

```

```

README.md: 100%                               536/536 [00:00<00:00, 21.3kB/s]

clean/validation-00000-of-00001.parquet: 100%                               9.19M/9.19M [00:00<00:00, 16.4MB/s]

Generating validation split: 100%                               73/73 [00:00<00:00, 296.61 examples/s]

Sampling rate: 16000

config.json:      2.28k/? [00:00<00:00, 79.9kB/s]

/usr/local/lib/python3.12/dist-packages/transformers/configuration_utils.py:335: UserWarning: Passing `gradient_checkpointing`
  warnings.warn(
model.safetensors: 100%                               1.27G/1.27G [00:27<00:00, 69.8MB/s]

Some weights of the model checkpoint at ehcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition were not used when initializing
- This IS expected if you are initializing Wav2Vec2ForSequenceClassification from the checkpoint of a model trained on another
- This IS NOT expected if you are initializing Wav2Vec2ForSequenceClassification from the checkpoint of a model that you expect
Some weights of Wav2Vec2ForSequenceClassification were not initialized from the model checkpoint at ehcalabres/wav2vec2-lg-xlsr
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Fetching 1 files: 100%                               1/1 [00:00<00:00, 4.66it/s]

preprocessor_config.json: 100%                               214/214 [00:00<00:00, 11.6kB/s]

Device set to use cuda:0
[{'score': 0.1323, 'label': 'calm'}, {'score': 0.1305, 'label': 'angry'}, {'score': 0.1267, 'label': 'sad'}, {'score': 0.1255,

```

The general process for using a model and feature extractor for audio classification is:

1. Instantiate a feature extractor and a model from the checkpoint name.
2. Process the audio signal to be classified with a feature extractor.
3. Pass the input through the model and take the `argmax` to retrieve the most likely class.
4. Convert the class id to a class name with `id2label` to return an interpretable result.

```

from transformers import AutoFeatureExtractor, AutoModelForAudioClassification
from datasets import load_dataset
import torch

dataset = load_dataset("hf-internal-testing/librispeech_asr_demo", "clean", split="validation")
dataset = dataset.sort("id")
sampling_rate = dataset.features["audio"].sampling_rate

feature_extractor = AutoFeatureExtractor.from_pretrained("superb/wav2vec2-base-superb-ks")
model = AutoModelForAudioClassification.from_pretrained("superb/wav2vec2-base-superb-ks")

inputs = feature_extractor(dataset[0]["audio"]["array"], sampling_rate=sampling_rate, return_tensors="pt")

with torch.no_grad():
    logits = model(**inputs).logits

predicted_class_ids = torch.argmax(logits, dim=-1).item()
predicted_label = model.config.id2label[predicted_class_ids]
predicted_label

```

```

Fetching 1 files: 100%                               1/1 [00:00<00:00, 7.07it/s]

preprocessor_config.json: 100%                               215/215 [00:00<00:00, 25.6kB/s]

config.json:      2.42k/? [00:00<00:00, 229kB/s]

pytorch_model.bin: 100%                               378M/378M [00:09<00:00, 46.1MB/s]

model.safetensors: 100%                               378M/378M [00:12<00:00, 23.1MB/s]

'_unknown_'

```

✓ Automatic speech recognition

Automatic speech recognition transcribes an audio signal to text. The [Common Voice](#) dataset is an example dataset that can be used for automatic speech recognition fine-tuning. It contains an audio file of a speaker and the corresponding sentence. If you'd like to fine-tune a model for automatic speech recognition, take a look at the [run_speech_recognition_ctc.py](#) or [run_speech_recognition_seq2seq.py](#) scripts or this [how-to guide](#).

The following examples demonstrate how to use a [pipeline\(\)](#) and a model and tokenizer for automatic speech recognition inference:

```
from transformers import pipeline
from datasets import load_dataset, Audio # Import Audio

dataset = load_dataset("hf-internal-testing/librispeech_asr_demo", "clean", split="validation")
dataset = dataset.sort("id")
dataset = dataset.cast_column("audio", Audio(decode=True)) # Cast to Audio with decode=True
audio_file = dataset[0]["audio"]["array"] # Access the array directly

speech_recognizer = pipeline(task="automatic-speech-recognition", model="facebook/wav2vec2-base-960h")
speech_recognizer(audio_file)
```

```
config.json:      1.60k/? [00:00<00:00, 40.1kB/s]

model.safetensors: 100%                               378M/378M [00:05<00:00, 83.9MB/s]
Some weights of Wav2Vec2ForCTC were not initialized from the model checkpoint at facebook/wav2vec2-base-960h and are newly init
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
tokenizer_config.json: 100%                             163/163 [00:00<00:00, 5.98kB/s]

vocab.json: 100%                                       291/291 [00:00<00:00, 14.2kB/s]

special_tokens_map.json: 100%                         85.0/85.0 [00:00<00:00, 4.03kB/s]

Fetching 1 files: 100%                                1/1 [00:00<00:00, 5.41it/s]

preprocessor_config.json: 100%                         159/159 [00:00<00:00, 5.43kB/s]

Device set to use cuda:0
{'text': 'MISTER QUILTER IS THE APOSTLE OF THE MIDDLE CLASSES AND WE ARE GLAD TO WELCOME HIS GOSPEL'}
```

The general process for using a model and processor for automatic speech recognition is:

1. Instantiate a processor (which regroups a feature extractor for input processing and a tokenizer for decoding) and a model from the checkpoint name.
2. Process the audio signal and text with a processor.
3. Pass the input through the model and take the `argmax` to retrieve the predicted text.
4. Decode the text with a tokenizer to obtain the transcription.

```
from transformers import AutoProcessor, AutoModelForCTC
from datasets import load_dataset
import torch

dataset = load_dataset("hf-internal-testing/librispeech_asr_demo", "clean", split="validation")
dataset = dataset.sort("id")
sampling_rate = dataset.features["audio"].sampling_rate

processor = AutoProcessor.from_pretrained("facebook/wav2vec2-base-960h")
model = AutoModelForCTC.from_pretrained("facebook/wav2vec2-base-960h")

inputs = processor(dataset[0]["audio"]["array"], sampling_rate=sampling_rate, return_tensors="pt")
with torch.no_grad():
    logits = model(**inputs).logits
predicted_ids = torch.argmax(logits, dim=-1)

transcription = processor.batch_decode(predicted_ids)
transcription[0]
```

```
Fetching 1 files: 100%                                1/1 [00:00<00:00, 43.01it/s]

Fetching 1 files: 100%                                1/1 [00:00<00:00, 21.36it/s]
```

```
Some weights of Wav2Vec2ForCTC were not initialized from the model checkpoint at facebook/wav2vec2-base-960h and are newly init
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
'MISTER QUILTER IS THE APOSTLE OF THE MIDDLE CLASSES AND WE ARE GLAD TO WELCOME HIS GOSPEL'
```

Image classification

Like text and audio classification, image classification assigns a class to an image. The [CIFAR-100](#) dataset is an example dataset that can be used for image classification fine-tuning. It contains an image and the corresponding class. If you'd like to fine-tune a model for image

classification, take a look at the [run_image_classification.py](#) script or this [how-to guide](#).

The following examples demonstrate how to use a [pipeline\(\)](#) and a model and tokenizer for image classification inference:

```
from transformers import pipeline

vision_classifier = pipeline(task="image-classification")
result = vision_classifier(
    inputs="https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/pipeline-cat-chonk.jpeg"
)
print("\n".join([f"Class {d['label']} with score {round(d['score'], 4)}" for d in result]))
```

No model was supplied, defaulted to google/vit-base-patch16-224 and revision 3f49326 (<https://huggingface.co/google/vit-base-patch16-224>). Using a pipeline without specifying a model name and revision in production is not recommended.

config.json: 69.7k/? [00:00<00:00, 1.39MB/s]

model.safetensors: 100% 346M/346M [00:07<00:00, 73.0MB/s]

Fetching 1 files: 100% 1/1 [00:00<00:00, 5.22it/s]

preprocessor_config.json: 100% 160/160 [00:00<00:00, 4.91kB/s]

Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is available for torch

Fetching 1 files: 100% 1/1 [00:00<00:00, 47.15it/s]

Device set to use cuda:0

Class lynx, catamount with score 0.4335

Class cougar, puma, catamount, mountain lion, painter, panther, Felis concolor with score 0.0348

Class snow leopard, ounce, Panthera uncia with score 0.0324

Class Egyptian cat with score 0.0239

Class tiger cat with score 0.0229

The general process for using a model and feature extractor for image classification is:

1. Instantiate a feature extractor and a model from the checkpoint name.
2. Process the image to be classified with a feature extractor.
3. Pass the input through the model and take the `argmax` to retrieve the predicted class.
4. Convert the class id to a class name with `id2label` to return an interpretable result.

```
from transformers import AutoFeatureExtractor, AutoModelForImageClassification
import torch
from datasets import load_dataset

dataset = load_dataset("huggingface/cats-image")
image = dataset["test"]["image"][0]

feature_extractor = AutoFeatureExtractor.from_pretrained("google/vit-base-patch16-224")
model = AutoModelForImageClassification.from_pretrained("google/vit-base-patch16-224")

inputs = feature_extractor(image, return_tensors="pt")

with torch.no_grad():
    logits = model(**inputs).logits

predicted_label = logits.argmax(-1).item()
print(model.config.id2label[predicted_label])
```

README.md: 100% 96.0/96.0 [00:00<00:00, 4.23kB/s]

cats_image.jpeg: 100% 173k/173k [00:00<00:00, 7.05MB/s]

Generating test split: 100% 1/1 [00:00<00:00, 17.32 examples/s]

Fetching 1 files: 100% 1/1 [00:00<00:00, 33.47it/s]

/usr/local/lib/python3.12/dist-packages/transformers/models/vit/feature_extraction_vit.py:30: FutureWarning: The class ViTFeatureExtraction is deprecated and will be removed in a future version. Please use ViTImageProcessor instead.
warnings.warn(
Egyptian cat